slopi-farm — гайд по миграции в Claude Code

Автономная ферма для решения task-based (jeopardy) CTF · рой DeepSeek V4.1-Flash через OpenRouter, эскалация в V4-Pro · полный хэндовер для старта с нуля
Составлено 2026-10-20 · репозиторий `/Users/karatinos/slopi-popi` (ветка master, без remote)

TL;DR для нового агента Code. Это оркестрированная «ферма» (рой дешёвых LLM-агентов + редкая эскалация), решающая CTF-задачи **двух типов**: офлайн-файлы (reverse/crypto/forensics) и живые инстансы (web/pwn). Один доменно-независимый движок + подключаемые профили. Код — в `slopi-popi/farm/`. **Платные прогоны нельзя запускать из облачной среды Claude** (egress-прокси режет внешние хосты) — только из **реального Terminal.app** пользователя двойным кликом по `.command`. Текущее действие: утром один клик по `START_COMPARE.command` даёт сравнение 4 прототипов (<$6). Главный рычаг качества — ACI/recon (сделано: 32/32 офлайн + зонд инстансов).

1. Как перенести проект в Claude Code

Код уже лежит на Mac пользователя в `/Users/karatinos/slopi-popi`. Claude Code работает с локальными папками напрямую, поэтому «миграция» — это открыть эту папку в Code и довезти то, чего нет в git. Два пути:

Путь A — локально (проще)	Открыть папку `/Users/karatinos/slopi-popi` в Claude Code. Всё на месте, включая секреты и кэш бенчмарка. Ничего никуда не копировать.
Путь B — через GitHub	Создать приватный репозиторий, `git remote add origin …`, `git push -u origin master`, затем в Code подключить/клонировать. **Секреты и кэш бенчмарка не в git** — перенести отдельно (см. §3, §6).

Что НЕ попадёт в git-клон (путь B) и должно быть перенесено руками:
- `farm/.secrets/ctfd.env`, `farm/.secrets/openrouter.env` — git-ignored (ключи);
- `farm/data/ctf_bench/files/` — 80 кэшированных файлов бенчмарка, **не закоммичены**; их надо скопировать или перекачать через `START_CTFD_PULL.command`.

Чек-лист старта в новой среде

1. Открыть `slopi-popi/farm/`.
2. Прочитать `farm/README.md` и этот гайд.
3. Восстановить `farm/.secrets/*.env` (§3).
4. Проверить наличие `farm/data/ctf_bench/files/` (80 файлов). Если пусто — `START_CTFD_PULL.command`.
5. `cd farm && python3 -m venv .venv && .venv/bin/pip install -r requirements.txt`.
6. Прогнать тесты (бесплатно): `python3 -m pytest -q` → должно быть **28 passed**.
7. Понять egress-ограничение (§2) — платное только из реального терминала.

2. Среда и КРИТИЧЕСКОЕ ограничение egress

И в текущем чат-интерфейсе, и в Claude Code агент исполняется в **облачном контейнере Anthropic** (+ опциональный мост к Mac пользователя). У контейнера **egress-allowlist**: наружу открыты только пакетные реестры (pip/npm) и GitHub. **OpenRouter и CTFd из контейнера/песочницы недостижимы.**

- Любой платный прогон (реальные вызовы модели) и обращение к живому CTFd **агент сам запустить не может**.
- Поэтому всё платное оформлено как **лаунчеры** `.command`, которые пользователь запускает двойным кликом в **настоящем Terminal.app** (там сеть есть, и там же его LAN до инстансов 192.168.1.108).
- Агент готовит лаунчеры и читает результаты (`data/runs/outcomes_*.json`) из общей папки.

Это ограничение **сохранится в Claude Code** — Code тоже облачный. Не планируй «сам прогоню и посмотрю»: готовь лаунчер, проси пользователя кликнуть.

3. Секреты и переменные окружения

Ключи лежат в `farm/.secrets/*.env` (git-ignored). Шлюз модели читает их из окружения. Имена переменных (значения восстановить из менеджера паролей / Codex-окружения пользователя):

farm/.secrets/openrouter.env	`OPENROUTER_API_KEY`, `DEEPSEEK_API_KEY` (= тот же ключ OpenRouter), `DEEPSEEK_BASE_URL` = `https://openrouter.ai/api/v1`, `DEEPSEEK_FLASH_MODEL` = `deepseek/deepseek-v4.1-flash`, `DEEPSEEK_THINK_MODEL`, `DEEPSEEK_PRO_MODEL` = `deepseek/deepseek-v4-pro`, `DEEPSEEK_REASONING_EFFORT` = `low`, `SLOPI_PROVIDER` = `deepseek`

farm/.secrets/ctfd.env	`CTFD_URL`, `CTFD_HEADER` (обход Authelia, заголовок `x-benchmark`), `CTFD_REGCODE` (код регистрации throwaway-аккаунтов)

`SLOPI_PROVIDER` переключает провайдера: `deepseek` (боевой, OpenRouter) · `mock` (детерминированный, $0, для тестов) · `scripted` (no-LLM автосолвер — валидатор конвейера, $0) · `auto`. Шаблон окружения — `farm/env.openrouter.sh` (ключ берёт из переменной, в репо ключа нет).

4. Архитектура: один движок, много профилей

Принцип (из блюпринтов в `../plan-of-work/`): выигрыш — не от «модель помощнее», а от **надёжности, координации и качества инструментов (ACI) ≈60-70% ценности**, плюс каскад дешёвых моделей с редкой эскалацией. Движок написан один раз и доменно-независим; всё «про CTF» — в профиле.

```
source ──► core: Engine → Scheduler(knapsack) → Ensemble → ReAct-agent → verdict/submit ──► Outcome
            gateway(Flash↔Pro)  sandbox(эфемерн.)  blackboard  governor(бюджет)
                 ▲ полиморфно вызывает ▲
      Profile:  ingest · toolset · system_prompt · verdict · submit
```

Пакет	Роль
`core/`	движок: `contracts` (Task/Candidate/Verdict/Outcome), `profile` (протокол+реестр), `toolset` (общий ACI-тулбелт + фабрика terminal-tool), `agent` (тонкий ReAct-цикл), `ensemble` (разнородные попытки, first-to-candidate / majority, blackboard, жёсткий дедлайн, consensus), `scheduler` (knapsack+admission+prefix-cache), `net` (reachability + **probe_instance**), `engine` (сборка+драйвер), `triage`, `report`
`profiles/`	`ctf` (сигнал успеха=флаг, сабмит на живой скорборд, playbook'и по категориям) и `generic` (ответ/программа, majority). Регистрируются по имени `@register`
`adapters/`	`CTFdSource` (живой скорборд: инстансы+файлы, реальная проверка, **per-prototype аккаунт**) и `LocalSource` (каталог задач; карантин файлов-решений, редакция gold)
`agent/`	`recon` (python-разведка: типы/энтропия/strings, pcap-сборка потоков, ELF-символы/импорты/.rodata), `image_inspect`
`gateway/`	шлюз модели: `providers` (DeepSeekProvider через OpenRouter, MockProvider, ScriptedSolverProvider), `client`, `types`
`sandbox/`	`factory` →docker/local; офлайн-корпус работает в LOCAL-фолбэке если docker недоступен
`pipeline/`	`verify` (канонизация флага), `triage`
`memory/`	`blackboard` (обмен находками между попытками), `solution_bank` + `embeddings` (RAG-банк), `store`
`observability/`	`governor` (губернатор бюджета, kill-switch), `metrics`, `charts`, `tracing`, `report`
`orchestration/`	`ctfd_client` (регистрация/логин/сабмит, ретраи)

5. Запуск, тесты, бенчмарк

```
# установка
cd farm && python3 -m venv .venv && .venv/bin/pip install -r requirements.txt

# что ферма умеет
python3 -m farm.cli profiles            # -> ctf, generic

# тесты (stdlib unittest/pytest, mock-провайдер, $0)
python3 -m pytest -q                     # 28 passed

# офлайн-прогон без денег (детерминированный автосолвер)
SLOPI_PROVIDER=scripted python3 -m farm.cli run --source local \
  --path research/clean_bench --answer-key data/answer_key_clean.json \
  --prototype p3 --limit 1.0 --label dryrun

# БОЕВОЙ прогон — ТОЛЬКО из реального Terminal.app (есть сеть)
python3 -m farm.cli run --source ctfd --base-url "$CTFD_URL" \
  --header "$CTFD_HEADER" --reg-code "$CTFD_REGCODE" \
  --profile ctf --prototype p3 --limit 1.5 --label live_p3
```

Лаунчеры (двойной клик в Terminal.app)

START_COMPARE.command	**главный** прогоняет p1/p3/p5/p7 по файлам+инстансам, каждый под своей учёткой, лимит $1.40/прототип (<$6), строит `data/runs/compare_live.{md,png}`
START_LIVE_RUN.command	одиночный прогон: `START_LIVE_RUN.command <proto> <cap$>`, оба режима
START_CTFD_PULL.command	обновить офлайн-кэш каталога CTFd
START_RUN.command	локальный прогон по каталогу задач

6. Бенчмарк и его целостность

- **Живой CTFd:** `https://ctfd.snowyx.dev` за прокси Authelia (обход заголовком `x-benchmark`). **72 задачи**: по 12 в crypto/forensics/misc/pwn/reverse/web. **32 офлайн (файлы) + 40 живых инстансов** (192.168.1.108:20xxx, доступны только из LAN пользователя).
- **Кэш:** `data/ctf_bench/` (каталог + 80 файлов, **не в git**), учётки в `data/ctf_bench/accounts.log`.
- **Ловушка целостности:** папки `../tasks-with-writeups/` (и местами `../tasks/`) содержат **РАЗБОРЫ с флагами и скрипты-солверы**, а не чистые задачи. Прямой прогон по ним даст **недостоверный** solve-rate. `LocalSource` это защищает: берёт условие только с 1-й стр. PDF, вырезает gold, карантинит файлы-решения (по имени и по тексту). Gold хранится для грейдинга в `data/answer_key*.json` — **никогда не стейджится агенту**.

7. Прототипы (ручки стратегии на одном движке = честный A/B)

#	ensemble	escalate F→Pro	recon	adaptive	consensus	смысл
p1	1	—	—	—	1	базовая линия (контроль)
p2	4	—	✓	—	1	ансамбль + recon
p3	3	✓	✓	—	1	умный каскад
p4	4	✓	✓	✓	1	каскад + адаптив
p5	5	—	✓	—	2	точность (self-consistency)
p6	2	✓	✓	✓	1	лёгкий каскад
p7	6	—	✓	✓	1	широкий дешёвый охват

Определены в `farm/cli.py:_PROTOTYPES`. Любую ручку можно переопределить флагом (`--ensemble/--escalate/--recon/--adaptive/--consensus`). Для сравнения выбраны p1/p3/p5/p7.

8. Конфиг и бюджеты (`farm/config.py`)

max_tokens_per_call	6144 — критично: V4.1-Flash «думает» по умолчанию, при 1024 отдавал пустоту
per_task_usd_cap	$1.20 (на задачу); `--limit` — жёсткий потолок на весь прогон
max_seconds	180 на траекторию; в ensemble есть **жёсткий общий дедлайн на задачу** (убил зависания 2-3 ч)
request_timeout / max_retries	45с / 1 (урезаны ради отзывчивости)
tool_timeout	50с на инструмент
hard_categories	`("pwn",)` — считаются сложными
PRICING / MODEL_IDS	тарифы OpenRouter: Flash $0.04/$0.144, Pro $0.209/$0.418 за 1M; модели из env

9. Нюансы и выстраданные уроки (ОБЯЗАТЕЛЬНО к прочтению)

- **ACI/recon — главный рычаг.** Офлайн-recon доведён до **32/32 задач high-signal** (strings ловит base64/hex/UUID-блобы; для stripped-ELF — импортируемые libc-вызовы + литералы .rodata; pcap — сборка TCP/UDP-потоков). Добавлен **instance recon** (`net.probe_instance`): HTTP (статус/заголовки/title/формы+поля/комментарии/robots) и raw-TCP баннер; web/pwn-playbook'и его используют.
- **Зависания.** Раньше ретраи модели + висящие коннекты давали прогоны по 2-3 ч/задача. Лечится жёстким дедлайном в ensemble + урезанными таймаутами. Не возвращай большие таймауты.
- **pcap endianness.** Была перепутана карта magic→endian — чинилось; при правках recon проверяй на Party Line/Records/Careless Beacon.
- **Сабмит флага.** Оборачивай в try/except + ретраи (ловит `httpx.RemoteProtocolError`, иначе падает весь прогон).
- **Per-prototype аккаунты.** Каждый прогон регистрирует отдельную учётку `slopi_<label>_<uid>` и пишет креды в `accounts.log` — раньше флаги уходили в анонимный аккаунт. Для каждого нового прототипа — своя учётка.
- **ScriptedSolver + endpoint-инъекция.** В `research/autosolve.py` НЕ должно быть `from __future__ import annotations` (ломало инъекцию `os.environ[...]` в начало файла).
- **Git на подключённой папке Mac.** Через мост удаление по умолчанию запрещено → git оставляет `.lock`/tmp-файлы, и следующие git-команды падают с «File exists». Решение: запросить у пользователя разрешение на удаление в папке (в Code — аналогичный механизм), затем чистить локи. Атрибуция коммитов: `Co-Authored-By: Claude …` (по системному напоминанию среды).
- **Реалистичные ожидания.** ИИ силён на web/forensics/шаблонном; слаб на hard-pwn и продвинутой крипте (NYU CTF Bench ~13.5% у EnIGMA). Цель — не 100%, а хороший cost-per-flag.

10. Текущее состояние и следующее действие

- **28/28 тестов зелёные**, рабочее дерево чистое, оба режима с recon, 4 лаунчера. 51 python-модуль.
- **Следующий шаг (за пользователем):** двойной клик `START_COMPARE.command` → реальные KPI по p1/p3/p5/p7 (solve-rate, $/флаг, медиана времени, эскалация), <$6, каждый под своей учёткой. По итогам — выбрать победителя и дожать полным прогоном.
- Дальнейший реальный прогресс требует живых данных прогона — не крути слепые офлайн-правки без обратной связи.

11. Указатели (читать в новой сессии)

farm/README.md	архитектура «один движок + профили», quickstart
../plan-of-work/	два исходных блюпринта (стратегия + инженерия), v1 (37 стр.) и v2 API-режим (16 стр.)
Проектные доки (claude.ai)	`claude/architecture-decisions.md` (ключевые решения, экономика, дорожная карта), `claude/implementation-status.md` (актуальный статус)
research/reports/	PDF-конспекты R&D и резюме сессий
NIGHT_REPORT_RU.pdf	отчёт последней автономной сессии